Advanced Architecture Patterns for Floosball

Overview

This document outlines advanced architecture patterns that could be implemented to further enhance the floosball football simulation. These patterns build upon the already-completed performance optimizations and service container architecture.

Current Architecture Foundation

✅ Already Implemented (Strong Foundation)

- **Service Container & Dependency Injection**: Centralized service management
- **Performance Optimizations**: Batched random generation, rating caching, optimized stats
- **API Response Builders**: Standardized, consistent API responses
- **Configuration Management**: Centralized config with caching and validation
- **Thread-Safe State Management**: Global state managed through service container

This solid foundation makes implementing advanced patterns much easier and more effective.

---

1. Event-Driven Architecture

Problem This Solves

Your current game simulation has tight coupling between game logic, stats tracking, API updates, and potential real-time features. When a play happens, the game directly calls stats methods, which directly update API state, etc.

Current State Example:

```
# Current - everything tightly coupled
def process_play(self, play_result):
    # Update game state
    self.homeScore += points
    self.totalPlays += 1

    # Direct calls to multiple systems
    player.addPassTd(yards, True)  # Player stats
    team.seasonTeamStats['Offense']['pts'] += points  # Team stats
    self.update_highlights(play_result)  # Highlights
    self.update_api_cache()  # API caching

    # Check game end
    if self.totalPlays >= GAME_MAX_PLAYS:
        self.endGame()
        self.notify_clients()  # WebSocket updates
```

Event-Driven Implementation:

Event Manager

```
from typing import Dict, List, Callable, Any
from dataclasses import dataclass
import asyncio

@dataclass
class GameEvent:
    """Represents a game event with data"""
    event_type: str
    game_id: str
    data: Dict[str, Any]
    timestamp: float = None

    def __post_init__(self):
        if self.timestamp is None:
            self.timestamp = time.time()

class GameEventManager:
    """Manages event publishing and subscriptions"""

    def __init__(self):
        self.subscribers: Dict[str, List[Callable]] = defaultdict(list)
        self.event_history: List[GameEvent] = []  # For debugging/replay

    def subscribe(self, event_type: str, handler: Callable):
        """Subscribe a handler to an event type"""
        self.subscribers[event_type].append(handler)

    async def publish(self, event: GameEvent):
        """Publish an event to all subscribers"""
        self.event_history.append(event)

        # Call all subscribers for this event type
        tasks = []
        for handler in self.subscribers[event.event_type]:
            if asyncio.iscoroutinefunction(handler):
                tasks.append(handler(event))
            else:
                # Run sync handlers in thread pool

tasks.append(asyncio.get_event_loop().run_in_executor(None, handler,
event))

        if tasks:
            await asyncio.gather(*tasks, return_exceptions=True)
```

Game Event Publishing

```python
class Game:
    def __init__(self, event_manager: GameEventManager):
        self.event_manager = event_manager

    async def process_play(self, play_result):
        # Update core game state only
        old_score = self.homeScore
        self.homeScore += play_result.points
        self.totalPlays += 1

        # Publish events - let other systems react
        await self.event_manager.publish(GameEvent(
            event_type='play_completed',
            game_id=self.id,
            data={
                'play': play_result.to_dict(),
                'game_state': {
                    'total_plays': self.totalPlays,
                    'plays_remaining': GAME_MAX_PLAYS - self.totalPlays,
                    'current_down': self.down,
                    'yards_to_go': self.yardsToFirstDown
                }
            }
        ))

        # Publish score change event if applicable
        if play_result.points > 0:
            await self.event_manager.publish(GameEvent(
                event_type='score_change',
                game_id=self.id,
                data={
                    'team': self.offensiveTeam.name,
                    'team_id': self.offensiveTeam.id,
                    'points_scored': play_result.points,
                    'old_score': old_score,
                    'new_score': self.homeScore,
                    'play_description': play_result.playText
                }
            ))

        # Check for game end
        if self.totalPlays >= GAME_MAX_PLAYS:
            await self.event_manager.publish(GameEvent(
                event_type='game_completed',
                game_id=self.id,
                data={
                    'final_score': {
                        'home': self.homeScore,
                        'away': self.awayScore
                    },
                    'winning_team': self.winningTeam.name if
self.winningTeam else None,
                    'total_plays': self.totalPlays
```

```
                }
            ))
```

Event Subscribers (Decoupled Systems)

```
class PlayerStatsEventHandler:
    """Handles player statistics updates via events"""

    def __init__(self, stats_service):
        self.stats_service = stats_service

    async def handle_play_completed(self, event: GameEvent):
        """Update player stats when play completes"""
        play_data = event.data['play']

        # Update player stats based on play type
        if play_data.get('passing_yards'):
            await self.stats_service.add_passing_yards(
                player_id=play_data['player_id'],
                yards=play_data['passing_yards'],
                is_td=play_data.get('is_td', False)
            )

class WebSocketEventHandler:
    """Handles real-time updates via WebSocket"""

    def __init__(self, websocket_manager):
        self.websocket_manager = websocket_manager

    async def handle_score_change(self, event: GameEvent):
        """Push score updates to connected clients"""
        await self.websocket_manager.broadcast_to_game_watchers(
            game_id=event.game_id,
            message={
                'type': 'score_update',
                'team': event.data['team'],
                'points': event.data['points_scored'],
                'new_score': event.data['new_score'],
                'play': event.data['play_description']
            }
        )

    async def handle_game_completed(self, event: GameEvent):
        """Push game completion notification"""
        await self.websocket_manager.broadcast_to_game_watchers(
            game_id=event.game_id,
            message={
                'type': 'game_final',
                'final_score': event.data['final_score'],
                'winner': event.data['winning_team']
            }
```

```
        )

class HighlightsEventHandler:
    """Automatically generates highlights from significant plays"""

    async def handle_play_completed(self, event: GameEvent):
        play_data = event.data['play']

        # Automatically detect highlight-worthy plays
        if (play_data.get('is_td') or
            play_data.get('yards', 0) >= 20 or
            play_data.get('is_interception') or
            play_data.get('is_fumble')):

            await self.add_to_highlights(event)

# Event system setup
event_manager = GameEventManager()

# Register handlers
stats_handler = PlayerStatsEventHandler(stats_service)
websocket_handler = WebSocketEventHandler(websocket_manager)
highlights_handler = HighlightsEventHandler()

event_manager.subscribe('play_completed',
stats_handler.handle_play_completed)
event_manager.subscribe('play_completed',
highlights_handler.handle_play_completed)
event_manager.subscribe('score_change',
websocket_handler.handle_score_change)
event_manager.subscribe('game_completed',
websocket_handler.handle_game_completed)
```

Benefits of Event-Driven Architecture

- **Decoupling**: Game logic doesn't know about WebSockets, stats, or highlights
- **Real-time Features**: WebSocket updates happen automatically when events fire
- **Extensibility**: Add new features by subscribing to existing events
- **Debugging**: Event history provides complete audit trail
- **Testing**: Mock event handlers to test each system independently
- **Performance**: Async event handling doesn't block game simulation

---

2. MVC (Model-View-Controller) Pattern

Problem This Solves

Your API endpoints currently mix data access, business logic, and response formatting in single functions. This makes testing difficult and violates separation of concerns.

Current State Example:

```python
# Everything mixed together
@app.get('/game/{id}')
async def get_game(id):
    # Data access mixed with business logic mixed with response formatting
    gameDict = {}
    for y in range(len(floosball.scheduleList)):
        weekGameList = floosball.scheduleList[y]
        for x in range(0,len(weekGameList['games'])):
            game = weekGameList['games'][x]
            if id == game.id:
                # Business logic
                win_prob = calculate_win_probability(game)

                # Response formatting
                gameDict['homeTeam'] = game.homeTeam.name
                gameDict['awayTeam'] = game.awayTeam.name
                gameDict['homeTeamWinProbability'] = round(win_prob * 100)
                # ... 50 more lines
                return gameDict
    return 'Game Not Found'
```

MVC Implementation:

Models (Data Access & Business Logic)

```python
from abc import ABC, abstractmethod
from typing import Optional, List

class GameRepository(ABC):
    """Abstract repository for game data access"""

    @abstractmethod
    def find_by_id(self, game_id: str) -> Optional['Game']:
        pass

    @abstractmethod
    def find_active_games(self) -> List['Game']:
        pass

    @abstractmethod
    def find_games_by_week(self, week: int) -> List['Game']:
        pass

class FloosballGameRepository(GameRepository):
    """Concrete implementation using existing floosball data"""

    def find_by_id(self, game_id: str) -> Optional['Game']:
        for week_games in floosball.scheduleList:
            for game in week_games['games']:
                if game.id == game_id:
```

```
                    return game
        return None

    def find_active_games(self) -> List['Game']:
        return floosball.activeSeason.activeGames

    def find_games_by_week(self, week: int) -> List['Game']:
        if week <= len(floosball.scheduleList):
            return floosball.scheduleList[week - 1]['games']
        return []

class GameModel:
    """Business logic for game operations"""

    def __init__(self, repository: GameRepository, rating_service):
        self.repository = repository
        self.rating_service = rating_service

    def get_game_by_id(self, game_id: str) -> Optional['Game']:
        """Get game with validation"""
        game = self.repository.find_by_id(game_id)
        if not game:
            raise GameNotFoundError(f"Game with id {game_id} not found")
        return game

    def calculate_win_probabilities(self, game: 'Game') -> tuple:
        """Calculate win probabilities with business logic"""
        # Use your existing calculation logic
        home_prob = game.homeTeamWinProbability
        away_prob = game.awayTeamWinProbability

        # Add business rules
        if game.isComplete:
            # If game is over, winner has 100% probability
            if game.homeScore > game.awayScore:
                home_prob, away_prob = 1.0, 0.0
            else:
                home_prob, away_prob = 0.0, 1.0

        return home_prob, away_prob

    def get_active_games_with_context(self) -> List['Game']:
        """Get active games with additional business context"""
        games = self.repository.find_active_games()

        # Add business logic - sort by importance
        return sorted(games, key=lambda g: (
            g.isPlayoffs,  # Playoff games first
            g.homeTeam.elo + g.awayTeam.elo,  # Then by combined ELO
            g.totalPlays  # Then by game progress
        ), reverse=True)

# Custom exceptions
```

```
class GameNotFoundError(Exception):
    pass
```

Views (Response Formatting)

```
class GameView:
    """Handles response formatting for game data"""

    def __init__(self, response_builder):
        self.response_builder = response_builder

    def render_game_detail(self, game: 'Game', win_probabilities: tuple) -
> dict:
        """Render detailed game information"""
        home_prob, away_prob = win_probabilities

        game_dict =
self.response_builder.build_game_with_probabilities(game)

        # Add view-specific enhancements
        game_dict.update({
            'homeTeamWinProbability': round(home_prob * 100),
            'awayTeamWinProbability': round(away_prob * 100),
            'gameProgress': f"{game.totalPlays}/{GAME_MAX_PLAYS}",
            'timeRemaining': GAME_MAX_PLAYS - game.totalPlays,
            'isCloseGame': abs(game.homeScore - game.awayScore) <= 7,
            'gameContext': self._get_game_context(game)
        })

        return game_dict

    def render_game_list(self, games: List['Game']) -> dict:
        """Render list of games"""
        return {
            'games': [self.response_builder.build_basic_game_dict(game)
for game in games],
            'count': len(games),
            'active_count': len([g for g in games if not g.isComplete])
        }

    def render_error(self, error: Exception, status_code: int = 400) ->
dict:
        """Render error response"""
        return {
            'error': True,
            'message': str(error),
            'error_type': error.__class__.__name__,
            'status_code': status_code
        }

    def _get_game_context(self, game: 'Game') -> dict:
```

```
        """Add contextual information about game importance"""
        context = {}

        if game.isPlayoffs:
            context['importance'] = 'playoff'
        elif game.homeTeam.clinchedPlayoffs or
game.awayTeam.clinchedPlayoffs:
            context['importance'] = 'playoff_implications'
        elif abs(game.homeTeam.seasonTeamStats['wins'] -
game.awayTeam.seasonTeamStats['wins']) <= 1:
            context['importance'] = 'close_record'
        else:
            context['importance'] = 'regular'

        return context
```

Controllers (Orchestration)

```
class GameController:
    """Orchestrates game operations between model and view"""

    def __init__(self, model: GameModel, view: GameView):
        self.model = model
        self.view = view

    async def get_game_detail(self, game_id: str) -> dict:
        """Get detailed game information"""
        try:
            # Use model for data and business logic
            game = self.model.get_game_by_id(game_id)
            win_probabilities =
self.model.calculate_win_probabilities(game)

            # Use view for response formatting
            return self.view.render_game_detail(game, win_probabilities)

        except GameNotFoundError as e:
            return self.view.render_error(e, 404)
        except Exception as e:
            logger.error(f"Error getting game {game_id}: {e}")
            return self.view.render_error(e, 500)

    async def get_active_games(self) -> dict:
        """Get all active games with context"""
        try:
            games = self.model.get_active_games_with_context()
            return self.view.render_game_list(games)

        except Exception as e:
            logger.error(f"Error getting active games: {e}")
            return self.view.render_error(e, 500)
```

```python
    async def get_games_by_week(self, week: int) -> dict:
        """Get games for specific week"""
        try:
            if week < 1 or week > len(floosball.scheduleList):
                raise ValueError(f"Invalid week: {week}")

            games = self.model.repository.find_games_by_week(week)
            return self.view.render_game_list(games)

        except ValueError as e:
            return self.view.render_error(e, 400)
        except Exception as e:
            logger.error(f"Error getting games for week {week}: {e}")
            return self.view.render_error(e, 500)

# Dependency injection setup
game_repository = FloosballGameRepository()
game_model = GameModel(game_repository, rating_service)
game_view = GameView(GameResponseBuilder())
game_controller = GameController(game_model, game_view)
```

API Endpoints (Thin Layer)

```python
# API endpoints become very thin
@app.get('/game/{id}')
async def get_game(id: str):
    """Get detailed game information"""
    return await game_controller.get_game_detail(id)

@app.get('/games/active')
async def get_active_games():
    """Get all currently active games"""
    return await game_controller.get_active_games()

@app.get('/games/week/{week}')
async def get_games_by_week(week: int):
    """Get games for specific week"""
    return await game_controller.get_games_by_week(week)
```

Benefits of MVC Pattern

- **Separation of Concerns**: Data, business logic, and presentation clearly separated
- **Testability**: Each layer can be unit tested independently
- **Reusability**: Same model can power web API, CLI tools, mobile apps
- **Maintainability**: Changes to response format don't affect business logic
- **Team Development**: Different developers can work on different layers

---

3. Plugin System Architecture

Problem This Solves

Your game rules are currently hardcoded. A plugin system would allow you to:

- Experiment with different game mechanics
- Support different league types (NFL, college, custom)
- A/B test rule changes
- Allow community extensions

Plugin Interface Design:

```python
from abc import ABC, abstractmethod
from typing import Dict, Any, Optional
from dataclasses import dataclass

@dataclass
class GameState:
    """Immutable game state snapshot"""
    down: int
    yards_to_go: int
    yards_to_endzone: int
    total_plays: int
    score_differential: int
    time_remaining: int  # plays remaining
    is_playoffs: bool
    game_type: str  # "nfl", "college", "custom"
    league_settings: Dict[str, Any]

@dataclass
class PlayContext:
    """Context for a specific play situation"""
    offensive_team_rating: int
    defensive_team_rating: int
    player_matchups: Dict[str, int]
    weather_conditions: Dict[str, Any]
    crowd_factor: int

class GameRulePlugin(ABC):
    """Base interface for all game rule plugins"""

    @property
    @abstractmethod
    def name(self) -> str:
        """Plugin name for identification"""
        pass

    @property
    @abstractmethod
    def priority(self) -> int:
        """Execution priority (higher = earlier)"""
```

```
        pass

    @abstractmethod
    def should_apply(self, game_state: GameState, context: PlayContext) ->
bool:
        """Check if this plugin should apply to current situation"""
        pass

    @abstractmethod
    def modify_play_outcome(self, game_state: GameState, context:
PlayContext,
                            base_outcome: Dict[str, Any]) -> Dict[str,
Any]:
        """Modify the play outcome based on plugin rules"""
        pass

    def setup(self, config: Dict[str, Any]) -> None:
        """Initialize plugin with configuration"""
        pass
```

Concrete Plugin Implementations:

```
class StandardNFLRules(GamePlugin):
    """Standard NFL game rules"""

    @property
    def name(self) -> str:
        return "standard_nfl"

    @property
    def priority(self) -> int:
        return 100  # Base rules run first

    def should_apply(self, game_state: GameState, context: PlayContext) ->
bool:
        return game_state.game_type == "nfl"

    def modify_play_outcome(self, game_state: GameState, context:
PlayContext,
                            base_outcome: Dict[str, Any]) -> Dict[str,
Any]:
        # Standard NFL rules
        outcome = base_outcome.copy()

        # 4th down rules
        if game_state.down == 4 and game_state.yards_to_go > 2:
            if game_state.yards_to_endzone < 35:  # Field goal range
                outcome['play_type'] = 'field_goal_attempt'
            else:
                outcome['play_type'] = 'punt'
```

```
        # Two-minute warning effects
        if game_state.time_remaining <= 4:  # 4 plays = ~2 minutes
            outcome['urgency_bonus'] = 2

        return outcome

class CollegeOvertimeRules(GameRulePlugin):
    """College football overtime rules"""

    @property
    def name(self) -> str:
        return "college_overtime"

    @property
    def priority(self) -> int:
        return 80

    def should_apply(self, game_state: GameState, context: PlayContext) ->
bool:
        return (game_state.game_type == "college" and
                game_state.total_plays > GAME_MAX_PLAYS)

    def modify_play_outcome(self, game_state: GameState, context:
PlayContext,
                            base_outcome: Dict[str, Any]) -> Dict[str,
Any]:
        outcome = base_outcome.copy()

        # College overtime: start at 25-yard line
        if game_state.total_plays == GAME_MAX_PLAYS + 1:
            outcome['field_position'] = 25

        # Must go for 2-point conversion after 2nd OT
        if game_state.total_plays > GAME_MAX_PLAYS + 12:  # 3rd OT
            outcome['force_two_point'] = True

        return outcome

class WeatherEffectsPlugin(GameRulePlugin):
    """Weather effects on gameplay"""

    @property
    def name(self) -> str:
        return "weather_effects"

    @property
    def priority(self) -> int:
        return 60

    def should_apply(self, game_state: GameState, context: PlayContext) ->
bool:
        return context.weather_conditions.get('severity', 0) > 0

    def modify_play_outcome(self, game_state: GameState, context:
```

```
PlayContext,
                            base_outcome: Dict[str, Any]) -> Dict[str,
Any]:
        outcome = base_outcome.copy()
        weather = context.weather_conditions

        if weather.get('rain', 0) > 5:  # Heavy rain
            outcome['fumble_chance'] *= 1.5
            outcome['passing_accuracy'] *= 0.9

        if weather.get('wind', 0) > 15:  # Strong wind
            outcome['kicking_accuracy'] *= 0.8
            outcome['long_pass_difficulty'] += 2

        if weather.get('temperature', 70) < 32:  # Freezing
            outcome['kicking_accuracy'] *= 0.9
            outcome['player_speed'] *= 0.95

        return outcome

class CustomLeagueRules(GameRulePlugin):
    """Custom rules for specific leagues"""

    def __init__(self):
        self.custom_rules = {}

    @property
    def name(self) -> str:
        return "custom_league"

    @property
    def priority(self) -> int:
        return 40  # Override other rules

    def setup(self, config: Dict[str, Any]) -> None:
        self.custom_rules = config.get('custom_rules', {})

    def should_apply(self, game_state: GameState, context: PlayContext) ->
bool:
        return game_state.league_settings.get('use_custom_rules', False)

    def modify_play_outcome(self, game_state: GameState, context:
PlayContext,
                            base_outcome: Dict[str, Any]) -> Dict[str,
Any]:
        outcome = base_outcome.copy()

        # Apply custom rules
        if 'explosive_plays_bonus' in self.custom_rules:
            if outcome.get('yards', 0) > 20:
                outcome['yards'] +=
self.custom_rules['explosive_plays_bonus']

        if 'underdog_boost' in self.custom_rules:
```

```
            if context.offensive_team_rating <
context.defensive_team_rating:
                outcome['success_bonus'] =
self.custom_rules['underdog_boost']

        return outcome
```

Plugin Manager:

```
class GameRuleEngine:
    """Manages and executes game rule plugins"""

    def __init__(self):
        self.plugins: List[GameRulePlugin] = []
        self.enabled_plugins: Set[str] = set()

    def register_plugin(self, plugin: GameRulePlugin, config: Dict[str,
Any] = None):
        """Register a new plugin"""
        plugin.setup(config or {})
        self.plugins.append(plugin)
        self.enabled_plugins.add(plugin.name)

        # Sort by priority
        self.plugins.sort(key=lambda p: p.priority, reverse=True)

        logger.info(f"Registered plugin: {plugin.name} (priority:
{plugin.priority})")

    def enable_plugin(self, plugin_name: str):
        """Enable a specific plugin"""
        self.enabled_plugins.add(plugin_name)

    def disable_plugin(self, plugin_name: str):
        """Disable a specific plugin"""
        self.enabled_plugins.discard(plugin_name)

    def process_play(self, game_state: GameState, context: PlayContext,
                     base_outcome: Dict[str, Any]) -> Dict[str, Any]:
        """Apply all applicable plugins to modify play outcome"""
        outcome = base_outcome.copy()

        for plugin in self.plugins:
            if (plugin.name in self.enabled_plugins and
                plugin.should_apply(game_state, context)):

                try:
                    outcome = plugin.modify_play_outcome(game_state,
context, outcome)
                    logger.debug(f"Applied plugin: {plugin.name}")
                except Exception as e:
```

```
                        logger.error(f"Error in plugin {plugin.name}: {e}")
                        # Continue with other plugins

        return outcome

    def get_active_plugins(self, game_state: GameState, context:
PlayContext) -> List[str]:
        """Get list of plugins that would apply to current situation"""
        active = []
        for plugin in self.plugins:
            if (plugin.name in self.enabled_plugins and
                plugin.should_apply(game_state, context)):
                active.append(plugin.name)
        return active
```

Integration with Game Engine:

```
class Game:
    def __init__(self, rule_engine: GameRuleEngine):
        self.rule_engine = rule_engine

    def process_play(self):
        # Build current game state
        game_state = GameState(
            down=self.down,
            yards_to_go=self.yardsToFirstDown,
            yards_to_endzone=self.yardsToEndzone,
            total_plays=self.totalPlays,
            score_differential=abs(self.homeScore - self.awayScore),
            time_remaining=GAME_MAX_PLAYS - self.totalPlays,
            is_playoffs=self.isPlayoffs,
            game_type=self.league_settings.get('type', 'nfl'),
            league_settings=self.league_settings
        )

        # Build play context
        context = PlayContext(
            offensive_team_rating=self.offensiveTeam.overallRating,
            defensive_team_rating=self.defensiveTeam.defenseRating,
            player_matchups=self.get_player_matchups(),
            weather_conditions=self.weather,
            crowd_factor=self.get_crowd_factor()
        )

        # Get base play outcome using existing logic
        base_outcome = self.simulate_base_play()

        # Apply plugins to modify outcome
        final_outcome = self.rule_engine.process_play(game_state, context,
base_outcome)
```

```
        # Execute the modified play
        self.execute_play_outcome(final_outcome)

# Setup different rule configurations
nfl_engine = GameRuleEngine()
nfl_engine.register_plugin(StandardNFLRules())
nfl_engine.register_plugin(WeatherEffectsPlugin())

college_engine = GameRuleEngine()
college_engine.register_plugin(StandardNFLRules())  # Base rules
college_engine.register_plugin(CollegeOvertimeRules())
college_engine.register_plugin(WeatherEffectsPlugin())

custom_engine = GameRuleEngine()
custom_engine.register_plugin(StandardNFLRules())
custom_engine.register_plugin(CustomLeagueRules(), {
    'custom_rules': {
        'explosive_plays_bonus': 5,
        'underdog_boost': 2
    }
})
```

Benefits of Plugin System

- **Extensibility**: Add new game modes without modifying core code
- **A/B Testing**: Test different rule variations easily
- **Configuration**: Different leagues can have completely different rules
- **Community**: Others can create and share rule plugins
- **Experimentation**: Try balance changes as plugins before committing

---

4. CQRS (Command Query Responsibility Segregation)

Problem This Solves

Your current system uses the same data structures and methods for both reading (queries) and writing (commands). This can lead to performance issues and coupling between different use cases.

Current State Example:

```
# Same methods handle reads and writes
class Player:
    def get_season_stats(self):
        # Expensive calculation every time
        return self.calculate_complex_season_stats()

    def add_passing_yards(self, yards):
        # Modifies same structure used for reads
        self.seasonStatsDict['passing']['yards'] += yards
        # Expensive recalculation
```

```
        self.recalculate_all_ratings()
        self.update_season_performance()
```

CQRS Implementation:

Command Side (Writes)

```
from dataclasses import dataclass
from typing import List
import json
from datetime import datetime

@dataclass
class Command:
    """Base command class"""
    timestamp: datetime
    user_id: Optional[str] = None

@dataclass
class AddPassingYards(Command):
    player_id: str
    yards: int
    is_touchdown: bool
    game_id: str

@dataclass
class UpdatePlayerRating(Command):
    player_id: str
    new_rating: int
    reason: str

class PlayerEvent:
    """Represents something that happened to a player"""
    def __init__(self, player_id: str, event_type: str, data: Dict[str,
Any]):
        self.player_id = player_id
        self.event_type = event_type
        self.data = data
        self.timestamp = datetime.now()
        self.event_id = str(uuid.uuid4())

class EventStore:
    """Stores all events that happen in the system"""

    def __init__(self):
        self.events: List[PlayerEvent] = []
        self.event_file = "events.jsonl"  # Append-only file

    def append_event(self, event: PlayerEvent):
        """Add new event to store"""
        self.events.append(event)
```

```python
        # Persist to file (append-only for durability)
        with open(self.event_file, 'a') as f:
            event_dict = {
                'event_id': event.event_id,
                'player_id': event.player_id,
                'event_type': event.event_type,
                'data': event.data,
                'timestamp': event.timestamp.isoformat()
            }
            f.write(json.dumps(event_dict) + '\n')

    def get_events_for_player(self, player_id: str) -> List[PlayerEvent]:
        """Get all events for a specific player"""
        return [e for e in self.events if e.player_id == player_id]

class PlayerCommandHandler:
    """Handles write operations for players"""

    def __init__(self, event_store: EventStore, event_bus):
        self.event_store = event_store
        self.event_bus = event_bus

    def handle_add_passing_yards(self, command: AddPassingYards):
        """Handle adding passing yards to a player"""
        # Create event
        event = PlayerEvent(
            player_id=command.player_id,
            event_type='passing_yards_added',
            data={
                'yards': command.yards,
                'is_touchdown': command.is_touchdown,
                'game_id': command.game_id,
                'command_timestamp': command.timestamp.isoformat()
            }
        )

        # Store event
        self.event_store.append_event(event)

        # Publish for read model updates
        self.event_bus.publish('player_stats_changed', {
            'player_id': command.player_id,
            'event_type': 'passing_yards_added',
            'event_data': event.data
        })

    def handle_update_rating(self, command: UpdatePlayerRating):
        """Handle player rating updates"""
        event = PlayerEvent(
            player_id=command.player_id,
            event_type='rating_updated',
            data={
                'new_rating': command.new_rating,
```

```
                'reason': command.reason,
                'command_timestamp': command.timestamp.isoformat()
            }
        )

        self.event_store.append_event(event)
        self.event_bus.publish('player_rating_changed', {
            'player_id': command.player_id,
            'new_rating': command.new_rating
        })
```

Query Side (Reads)

```
@dataclass
class PlayerSeasonStatsReadModel:
    """Optimized read model for season stats"""
    player_id: str
    season: int
    passing_yards: int = 0
    passing_tds: int = 0
    passing_attempts: int = 0
    rushing_yards: int = 0
    rushing_tds: int = 0
    receiving_yards: int = 0
    receiving_tds: int = 0
    fantasy_points: int = 0
    games_played: int = 0

    # Pre-calculated derived values
    yards_per_attempt: float = 0.0
    yards_per_game: float = 0.0
    touchdown_rate: float = 0.0

    last_updated: datetime = None

@dataclass
class PlayerCareerStatsReadModel:
    """Optimized read model for career stats"""
    player_id: str
    total_seasons: int = 0
    career_passing_yards: int = 0
    career_tds: int = 0
    career_games: int = 0
    best_season_yards: int = 0
    best_season_year: int = 0

    # Pre-calculated career averages
    avg_yards_per_season: float = 0.0
    avg_tds_per_season: float = 0.0

    last_updated: datetime = None
```

```
class PlayerQueryHandler:
    """Handles read operations for players (optimized for queries)"""

    def __init__(self):
        self.season_stats: Dict[str, PlayerSeasonStatsReadModel] = {}
        self.career_stats: Dict[str, PlayerCareerStatsReadModel] = {}
        self.rating_cache: Dict[str, int] = {}

    def get_season_stats(self, player_id: str, season: int = None) ->
PlayerSeasonStatsReadModel:
        """Get season stats (instant lookup)"""
        if season is None:
            season = floosball.activeSeason.currentSeason

        key = f"{player_id}_{season}"
        if key not in self.season_stats:
            # Initialize empty read model
            self.season_stats[key] = PlayerSeasonStatsReadModel(
                player_id=player_id,
                season=season
            )

        return self.season_stats[key]

    def get_career_stats(self, player_id: str) ->
PlayerCareerStatsReadModel:
        """Get career stats (instant lookup)"""
        if player_id not in self.career_stats:
            self.career_stats[player_id] = PlayerCareerStatsReadModel(
                player_id=player_id
            )

        return self.career_stats[player_id]

    def get_top_players_by_yards(self, limit: int = 10) ->
List[PlayerSeasonStatsReadModel]:
        """Get top players by passing yards (pre-sorted)"""
        current_season = floosball.activeSeason.currentSeason
        season_players = [
            stats for stats in self.season_stats.values()
            if stats.season == current_season
        ]

        return sorted(season_players,
                      key=lambda x: x.passing_yards,
                      reverse=True)[:limit]

    def handle_player_stats_changed(self, event_data):
        """Update read models when write operations occur"""
        player_id = event_data['player_id']
        event_type = event_data['event_type']
        data = event_data['event_data']
```

```
        if event_type == 'passing_yards_added':
            # Update season stats read model
            season_stats = self.get_season_stats(player_id)
            season_stats.passing_yards += data['yards']
            season_stats.passing_attempts += 1

            if data['is_touchdown']:
                season_stats.passing_tds += 1
                season_stats.fantasy_points += 6

            # Recalculate derived values
            if season_stats.passing_attempts > 0:
                season_stats.yards_per_attempt = (
                    season_stats.passing_yards /
season_stats.passing_attempts
                )

            if season_stats.games_played > 0:
                season_stats.yards_per_game = (
                    season_stats.passing_yards / season_stats.games_played
                )

            season_stats.last_updated = datetime.now()

            # Update career stats
            career_stats = self.get_career_stats(player_id)
            career_stats.career_passing_yards += data['yards']
            if data['is_touchdown']:
                career_stats.career_tds += 1

            # Update best season if needed
            if season_stats.passing_yards >
career_stats.best_season_yards:
                career_stats.best_season_yards =
season_stats.passing_yards
                career_stats.best_season_year = season_stats.season

            career_stats.last_updated = datetime.now()

    def handle_player_rating_changed(self, event_data):
        """Update rating cache"""
        self.rating_cache[event_data['player_id']] =
event_data['new_rating']
```

Integration Layer

```
class PlayerService:
    """Unified interface that routes to command or query handlers"""

    def __init__(self, command_handler: PlayerCommandHandler,
                 query_handler: PlayerQueryHandler):
```

```
        self.commands = command_handler
        self.queries = query_handler

    # Write operations go to command side
    def add_passing_yards(self, player_id: str, yards: int, is_td: bool,
game_id: str):
        """Add passing yards (write operation)"""
        command = AddPassingYards(
            player_id=player_id,
            yards=yards,
            is_touchdown=is_td,
            game_id=game_id,
            timestamp=datetime.now()
        )
        self.commands.handle_add_passing_yards(command)

    def update_player_rating(self, player_id: str, new_rating: int,
reason: str):
        """Update player rating (write operation)"""
        command = UpdatePlayerRating(
            player_id=player_id,
            new_rating=new_rating,
            reason=reason,
            timestamp=datetime.now()
        )
        self.commands.handle_update_rating(command)

    # Read operations go to query side
    def get_season_stats(self, player_id: str) ->
PlayerSeasonStatsReadModel:
        """Get season stats (read operation - instant!)"""
        return self.queries.get_season_stats(player_id)

    def get_career_stats(self, player_id: str) ->
PlayerCareerStatsReadModel:
        """Get career stats (read operation - instant!)"""
        return self.queries.get_career_stats(player_id)

    def get_leaderboard(self) -> List[PlayerSeasonStatsReadModel]:
        """Get leaderboard (read operation - instant!)"""
        return self.queries.get_top_players_by_yards()

# Setup
event_store = EventStore()
event_bus = EventBus()
command_handler = PlayerCommandHandler(event_store, event_bus)
query_handler = PlayerQueryHandler()

# Wire up event handlers
event_bus.subscribe('player_stats_changed',
query_handler.handle_player_stats_changed)
event_bus.subscribe('player_rating_changed',
query_handler.handle_player_rating_changed)
```

```
player_service = PlayerService(command_handler, query_handler)
```

Benefits of CQRS

- **Performance**: Reads are instant (pre-calculated read models)
- **Scalability**: Read and write databases can be scaled independently
- **Flexibility**: Different data models optimized for different use cases
- **Audit Trail**: Complete history of all changes in event store
- **Eventual Consistency**: Writes don't block reads, system stays responsive

---

5. Microservices Architecture

Problem This Solves

Your current application is a monolith where everything runs in one process. As the application grows, this can lead to:

- Scaling bottlenecks (can't scale game simulation separately from stats)
- Technology lock-in (everything must use same language/database)
- Deployment complexity (any change requires full redeployment)

Current Monolithic State:

```
floosball_api.py (1200+ lines)
├── Game simulation
├── Player stats
├── Team management
├── API endpoints
├── Real-time updates
└── Data persistence
```

Microservices Architecture:

Game Simulation Service

```python
# game_simulation_service.py
from fastapi import FastAPI
from typing import Dict, Any

app = FastAPI(title="Game Simulation Service")

class GameSimulationService:
    """Handles only game simulation logic"""

    def __init__(self, event_bus, rule_engine):
```

```
        self.event_bus = event_bus
        self.rule_engine = rule_engine

    async def simulate_play(self, game_state: Dict[str, Any]) -> Dict[str,
Any]:
        """Simulate a single play"""
        # Pure game simulation logic
        play_result = self.rule_engine.process_play(game_state)

        # Publish events for other services
        await self.event_bus.publish('play_completed', {
            'game_id': game_state['game_id'],
            'play_result': play_result,
            'new_game_state': self.calculate_new_state(game_state,
play_result)
        })

        return play_result

    async def simulate_full_game(self, game_config: Dict[str, Any]) ->
Dict[str, Any]:
        """Simulate an entire game"""
        game_id = game_config['game_id']
        game_state = self.initialize_game_state(game_config)

        plays = []
        while not game_state['is_complete']:
            play_result = await self.simulate_play(game_state)
            plays.append(play_result)
            game_state = self.update_game_state(game_state, play_result)

        # Publish game completion
        await self.event_bus.publish('game_completed', {
            'game_id': game_id,
            'final_state': game_state,
            'all_plays': plays
        })

        return game_state

@app.post('/simulate-play')
async def simulate_play(game_state: Dict[str, Any]):
    return await simulation_service.simulate_play(game_state)

@app.post('/simulate-game')
async def simulate_game(game_config: Dict[str, Any]):
    return await simulation_service.simulate_full_game(game_config)
```

Player Stats Service

```python
# player_stats_service.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI(title="Player Stats Service")

class StatsUpdate(BaseModel):
    player_id: str
    stat_type: str
    value: int
    game_id: str

class PlayerStatsService:
    """Handles only player statistics"""

    def __init__(self, stats_repository):
        self.stats_repository = stats_repository

    async def update_stats(self, stats_update: StatsUpdate):
        """Update player statistics"""
        await self.stats_repository.update_player_stat(
            player_id=stats_update.player_id,
            stat_type=stats_update.stat_type,
            value=stats_update.value,
            game_id=stats_update.game_id
        )

    async def get_player_stats(self, player_id: str, season: int = None):
        """Get comprehensive player stats"""
        return await self.stats_repository.get_player_stats(player_id,
season)

    async def get_leaderboard(self, stat_type: str, limit: int = 50):
        """Get leaderboard for specific stat"""
        return await self.stats_repository.get_leaderboard(stat_type,
limit)

    async def handle_play_completed(self, event_data):
        """Handle play completion events from game simulation"""
        play_result = event_data['play_result']

        # Update relevant player stats
        for player_stat in play_result.get('player_stats', []):
            await self.update_stats(StatsUpdate(
                player_id=player_stat['player_id'],
                stat_type=player_stat['stat_type'],
                value=player_stat['value'],
                game_id=event_data['game_id']
            ))

@app.post('/update-stats')
async def update_stats(stats_update: StatsUpdate):
    await stats_service.update_stats(stats_update)
```

```
    return {'success': True}

@app.get('/player/{player_id}/stats')
async def get_player_stats(player_id: str, season: int = None):
    try:
        stats = await stats_service.get_player_stats(player_id, season)
        if not stats:
            raise HTTPException(status_code=404, detail="Player not
found")
        return stats
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get('/leaderboard/{stat_type}')
async def get_leaderboard(stat_type: str, limit: int = 50):
    return await stats_service.get_leaderboard(stat_type, limit)
```

Ratings Calculation Service

```
# ratings_service.py
from fastapi import FastAPI

app = FastAPI(title="Ratings Service")

class RatingsService:
    """Handles only rating calculations"""

    def __init__(self, rating_cache):
        self.rating_cache = rating_cache

    async def calculate_player_rating(self, player_id: str, attributes:
Dict[str, Any]) -> int:
        """Calculate player rating with caching"""
        cached_rating = self.rating_cache.get_cached_rating(
            f"player_{player_id}", attributes
        )

        if cached_rating:
            return cached_rating

        # Expensive calculation
        rating = self._perform_rating_calculation(attributes)

        # Cache the result
        self.rating_cache.cache_rating(f"player_{player_id}", rating,
attributes)

        return rating

    async def calculate_team_rating(self, team_id: str) -> int:
        """Calculate team rating based on players"""
```

```
        # Use the existing rating cache we implemented
        return await self._calculate_team_composite_rating(team_id)

    async def get_rating_distribution(self) -> Dict[str, int]:
        """Get distribution of player ratings"""
        return await self._analyze_rating_distribution()

@app.get('/player/{player_id}/rating')
async def get_player_rating(player_id: str):
    # Get player attributes from player service
    attributes = await
player_service_client.get_player_attributes(player_id)
    rating = await ratings_service.calculate_player_rating(player_id,
attributes)
    return {'player_id': player_id, 'rating': rating}

@app.get('/team/{team_id}/rating')
async def get_team_rating(team_id: str):
    rating = await ratings_service.calculate_team_rating(team_id)
    return {'team_id': team_id, 'rating': rating}
```

API Gateway (Orchestrates Everything)

```
# api_gateway.py
from fastapi import FastAPI
import httpx
from typing import Dict, Any

app = FastAPI(title="Floosball API Gateway")

class ServiceClient:
    """Client for communicating with microservices"""

    def __init__(self):
        self.game_service_url = "http://game-simulation:8001"
        self.stats_service_url = "http://player-stats:8002"
        self.ratings_service_url = "http://ratings:8003"

    async def get_game_data(self, game_id: str):
        async with httpx.AsyncClient() as client:
            response = await client.get(f"
{self.game_service_url}/game/{game_id}")
            return response.json()

    async def get_player_stats(self, player_id: str):
        async with httpx.AsyncClient() as client:
            response = await client.get(f"
{self.stats_service_url}/player/{player_id}/stats")
            return response.json()

    async def get_player_rating(self, player_id: str):
```

```
        async with httpx.AsyncClient() as client:
            response = await client.get(f"
{self.ratings_service_url}/player/{player_id}/rating")
            return response.json()

service_client = ServiceClient()

@app.get('/game/{game_id}/complete')
async def get_complete_game_data(game_id: str):
    """Aggregate data from multiple services"""

    # Call multiple services in parallel
    game_data, stats_data, ratings_data = await asyncio.gather(
        service_client.get_game_data(game_id),
        service_client.get_game_stats(game_id),
        service_client.get_game_team_ratings(game_id),
        return_exceptions=True
    )

    return {
        'game': game_data,
        'stats': stats_data,
        'ratings': ratings_data
    }

@app.get('/player/{player_id}/profile')
async def get_complete_player_profile(player_id: str):
    """Get comprehensive player information"""

    # Aggregate from multiple services
    stats, rating = await asyncio.gather(
        service_client.get_player_stats(player_id),
        service_client.get_player_rating(player_id)
    )

    return {
        'player_id': player_id,
        'current_stats': stats,
        'current_rating': rating['rating'],
        'profile_complete': True
    }
```

Service Communication (Event Bus)

```
# event_bus.py (shared between services)
import asyncio
import json
from typing import Dict, Any, Callable, List

class DistributedEventBus:
    """Event bus for microservices communication"""
```

```python
    def __init__(self, redis_client=None):
        self.redis = redis_client  # For production
        self.local_subscribers = defaultdict(list)  # For development

    async def publish(self, event_type: str, event_data: Dict[str, Any]):
        """Publish event to all interested services"""
        event = {
            'type': event_type,
            'data': event_data,
            'timestamp': datetime.now().isoformat()
        }

        if self.redis:
            # Production: use Redis pub/sub
            await self.redis.publish(event_type, json.dumps(event))
        else:
            # Development: local event handling
            for handler in self.local_subscribers[event_type]:
                await handler(event)

    def subscribe(self, event_type: str, handler: Callable):
        """Subscribe to event type"""
        self.local_subscribers[event_type].append(handler)
```

Docker Compose Setup:

```yaml
# docker-compose.yml
version: '3.8'

services:
  game-simulation:
    build: ./services/game-simulation
    ports:
      - "8001:8000"
    environment:
      - SERVICE_NAME=game-simulation
      - REDIS_URL=redis://redis:6379
    depends_on:
      - redis

  player-stats:
    build: ./services/player-stats
    ports:
      - "8002:8000"
    environment:
      - SERVICE_NAME=player-stats
      - DATABASE_URL=postgresql://user:pass@postgres:5432/stats
    depends_on:
      - postgres
      - redis
```

```yaml
  ratings:
    build: ./services/ratings
    ports:
      - "8003:8000"
    environment:
      - SERVICE_NAME=ratings
      - REDIS_URL=redis://redis:6379
    depends_on:
      - redis

  api-gateway:
    build: ./services/api-gateway
    ports:
      - "8000:8000"
    environment:
      - GAME_SERVICE_URL=http://game-simulation:8000
      - STATS_SERVICE_URL=http://player-stats:8000
      - RATINGS_SERVICE_URL=http://ratings:8000
    depends_on:
      - game-simulation
      - player-stats
      - ratings

  redis:
    image: redis:alpine
    ports:
      - "6379:6379"

  postgres:
    image: postgres:13
    environment:
      - POSTGRES_DB=floosball
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password
    ports:
      - "5432:5432"
```

Benefits of Microservices

- **Independent Scaling**: Scale game simulation separately from stats
- **Technology Freedom**: Use different languages/databases for different services
- **Team Independence**: Different teams can work on different services
- **Fault Isolation**: If ratings service fails, game simulation still works
- **Deployment Flexibility**: Deploy services independently

Drawbacks to Consider

- **Complexity**: Network calls, service discovery, distributed debugging
- **Latency**: Network calls are slower than in-memory calls
- **Data Consistency**: Managing transactions across services is complex

- **Operational Overhead**: More services to monitor and deploy

---

Implementation Recommendations

Priority Order for Your Floosball Application:

Phase 1: Event-Driven Architecture (Highest Value)

- **Why First**: Enables real-time WebSocket features with minimal changes
- **Implementation**: Add event system to existing monolith
- **Benefits**: Decouples game logic from stats/updates, enables real-time features
- **Effort**: Medium (2-3 weeks)

Phase 2: Plugin System (High Value, Fun Factor)

- **Why Second**: Allows experimentation with game rules and mechanics
- **Implementation**: Extract rule logic into pluggable components
- **Benefits**: A/B testing, custom leagues, community extensions
- **Effort**: Medium (2-4 weeks)

Phase 3: MVC Pattern (Organizational Benefits)

- **Why Third**: Improves code organization as API grows
- **Implementation**: Refactor existing API endpoints gradually
- **Benefits**: Better testability, cleaner separation of concerns
- **Effort**: Low-Medium (1-2 weeks per major endpoint group)

Future Considerations:

Phase 4: CQRS (Performance at Scale)

- **When**: When you have performance issues with complex queries
- **Implementation**: Start with read models for leaderboards and stats
- **Benefits**: Much faster queries, better scalability
- **Effort**: High (4-6 weeks)

Phase 5: Microservices (Scale & Team Growth)

- **When**: When you need independent scaling or have multiple developers
- **Implementation**: Extract services one at a time (stats service first)
- **Benefits**: Independent scaling, technology diversity, team independence
- **Effort**: Very High (8-12 weeks for full migration)

Quick Win Opportunities:

1. **Start with Events**: Add event publishing to existing game simulation
2. **Build Plugin Framework**: Extract one rule type (e.g., overtime rules) as proof of concept
3. **Refactor One API Group**: Pick player endpoints and apply MVC pattern

The beauty of these patterns is that they can be implemented incrementally and build upon each other! 🚀

Conclusion

These advanced architecture patterns represent the next level of evolution for your floosball application. With your already-solid foundation of performance optimizations and service container architecture, implementing these patterns would be much more straightforward than starting from scratch.

The event-driven architecture in particular would unlock real-time features like WebSocket updates, while the plugin system would make the application much more fun to experiment with and extend.

Each pattern solves specific problems and provides specific benefits, so they can be adopted based on your current needs and priorities.